AWS / AWS NETWORKING

VPC Peering

Connect two VPCs. Prove private connectivity.

EWB Cloud Labs - Student Project Workbook (Project 01)

2 VPCs / 3 EC2 instances / 1 private peering connection

Project report

Name:	Naman Rathore
Batch:	AWS Internship Batch 1
Project title:	AWS VPC Peering (EC2, route tables, security groups, bastion)
Region:	Mumbai (ap-south-1)
Completion date:	24 October 2026

1. Project overview and architecture

EWB has an application in VPC A and a second workload in VPC B. Both application servers must stay private. I created a VPC peering connection so the two servers exchange traffic over their private IPv4 addresses, and used one public bastion host in VPC A for administration (SSH ProxyJump from my laptop). VPC B has no internet gateway, and neither application server has a public IP.

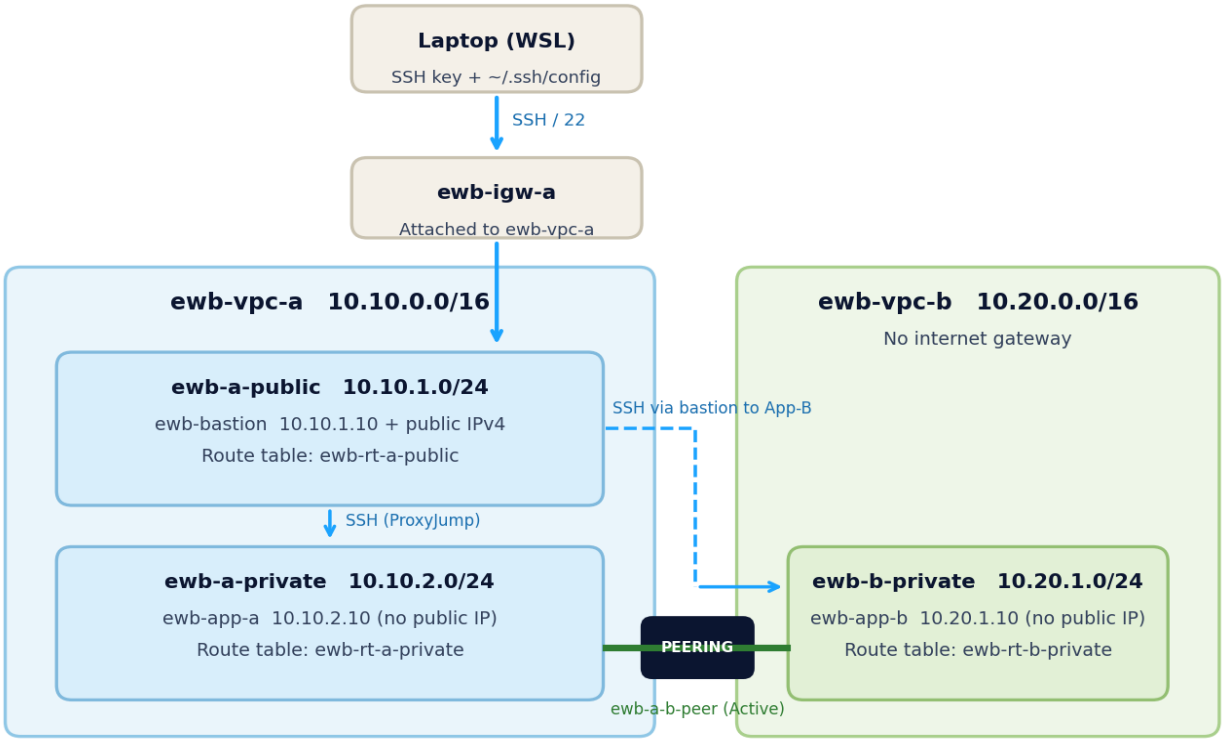


Figure 1 - Architecture. Administration: laptop > bastion > App-A or App-B. Project traffic: App-A <> App-B over private IPs through the peering connection.

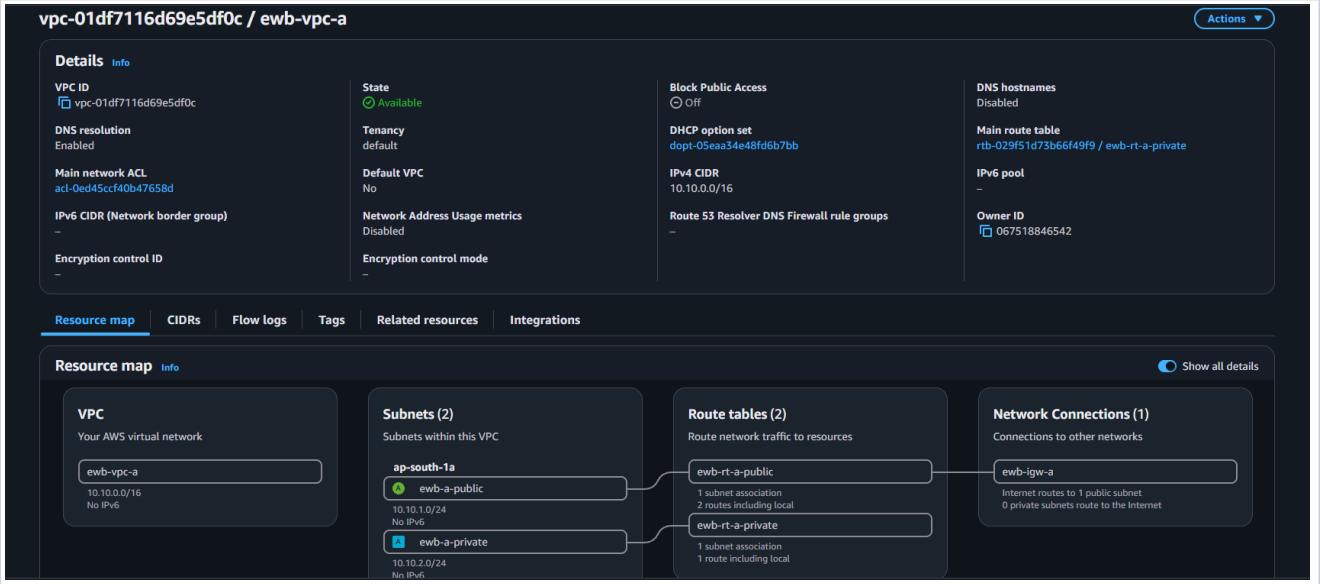
Resource inventory (actual IDs from my AWS account)

Resource	Name tag	ID	CIDR / IP
VPC A	ewb-vpc-a	`vpc-01df7116d69e5df0c`	10.10.0.0/16
Subnet (public)	ewb-a-public	`subnet-09fe796537383b1df`	10.10.1.0/24
Subnet (private)	ewb-a-private	`subnet-0f7c4601f41c543e1`	10.10.2.0/24
VPC B	ewb-vpc-b	`vpc-0f51163cd45441b8b`	10.20.0.0/16
Subnet (private)	ewb-b-private	`subnet-03a1ec0287c193729`	10.20.1.0/24
Internet gateway	ewb-igw-a	`igw-0bab6c17b7aaf194e`	Attached to ewb-vpc-a
Route table	ewb-rt-a-public	`rtb-0e24c9f36dc449eb3`	assoc: ewb-a-public
Route table	ewb-rt-a-private	`rtb-029f51d73b66f49f9`	assoc: ewb-a-private
Route table	ewb-rt-b-private	`rtb-0acc77e9fbbf10890`	assoc: ewb-b-private
Security group	ewb-sg-bastion	`sg-06e7870113e6d355b`	VPC A
Security group	ewb-sg-app-a	`sg-0bff98334e4b83efc`	VPC A
Security group	ewb-sg-app-b	`sg-07e360092b44cd742`	VPC B
Peering connection	ewb-a-b-peer	`pcx-0d6b0d880bdd17250`	Active
EC2 instance	ewb-bastion	`i-091594a27deb793d8`	10.10.1.10 + public 13.233.190.31
EC2 instance	ewb-app-a	`i-0e4795cc8e98e1a79`	10.10.2.10 (no public IP)
EC2 instance	ewb-app-b	`i-06e4b5758fe592e80`	10.20.1.10 (no public IP)

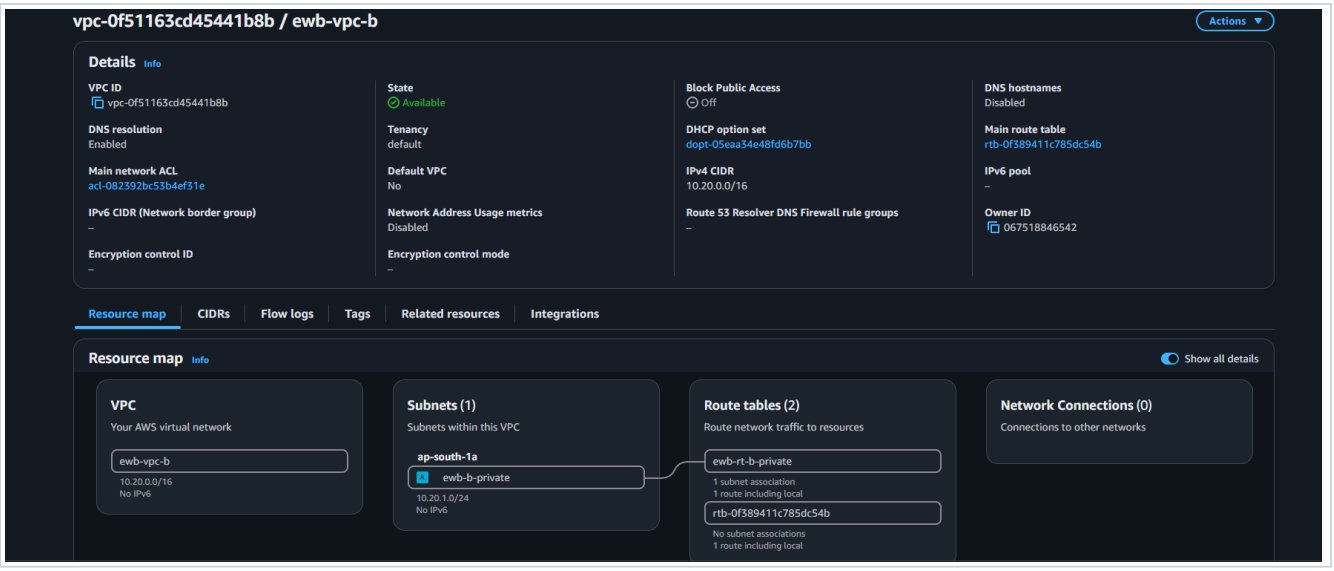
Key pair: ewb-lab (RSA). Instances: Amazon Linux 2023, t3.micro. 10.10.0.0/16 and 10.20.0.0/16 do not overlap, which is required for VPC peering.

2. VPCs, subnets, internet gateway and route tables

VPC A contains one public and one private subnet. The public route table has a default route to the internet gateway; the private route table has no internet route. VPC B has one private subnet and no internet gateway.

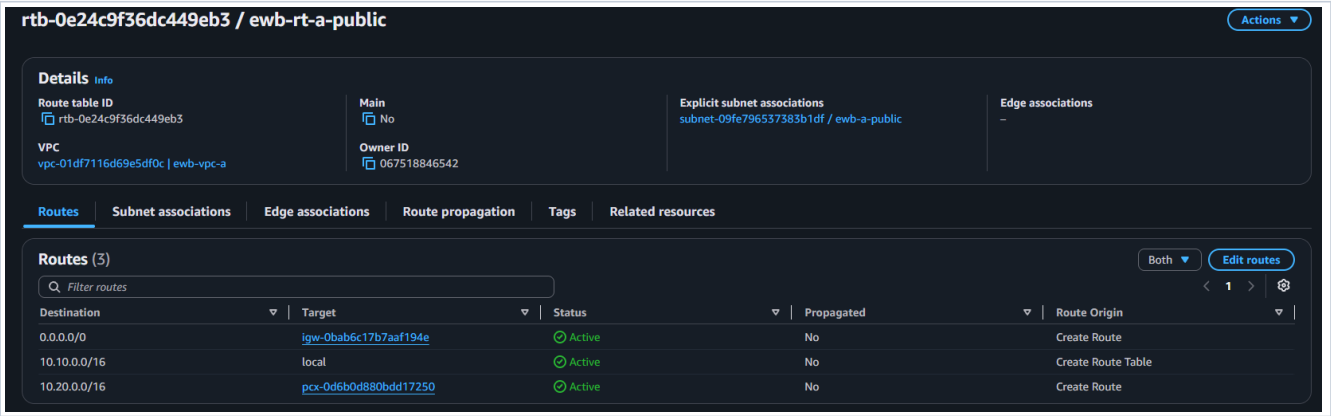


VPC A (ewb-vpc-a, 10.10.0.0/16) - resource map: subnets ewb-a-public 10.10.1.0/24 and ewb-a-private 10.10.2.0/24, route tables ewb-rt-a-public (2 routes incl. local) and ewb-rt-a-private, and internet gateway ewb-igw-a.

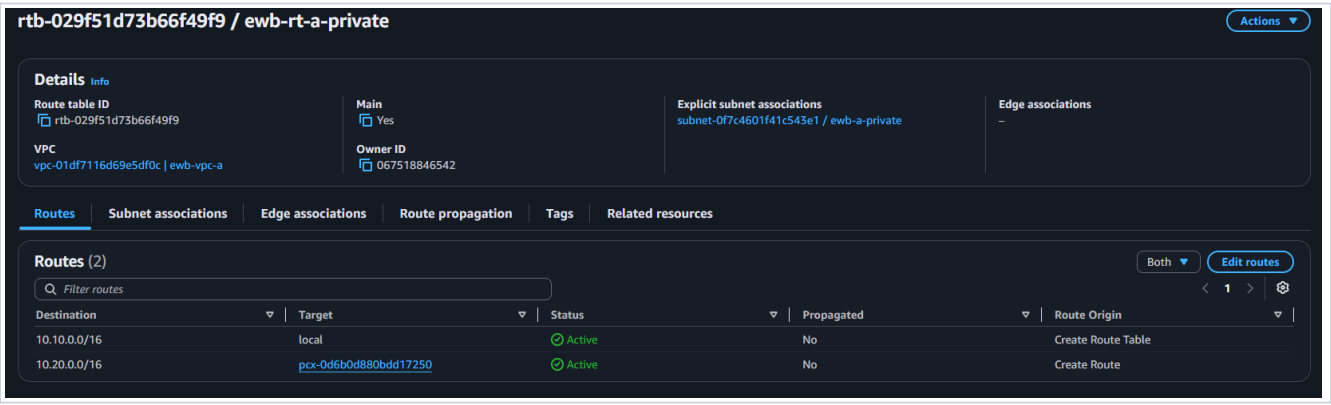


VPC B (ewb-vpc-b, 10.20.0.0/16) - resource map: subnet ewb-b-private 10.20.1.0/24 associated with ewb-rt-b-private; no network connections (no internet gateway).

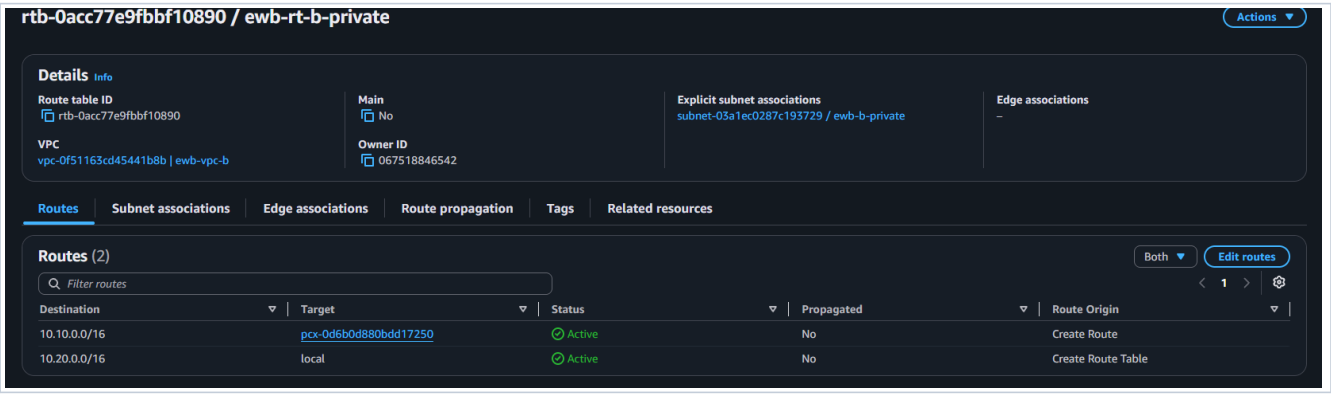
Route tables and subnet associations (after peering routes were added)



ewb-rt-a-public - associated with ewb-a-public. Routes: 0.0.0.0/0 to igw (bastion internet access), 10.10.0.0/16 local, 10.20.0.0/16 to pcx-0d6b0d880bdd17250 (needed because ProxyJump makes the bastion open the SSH connection to App-B).

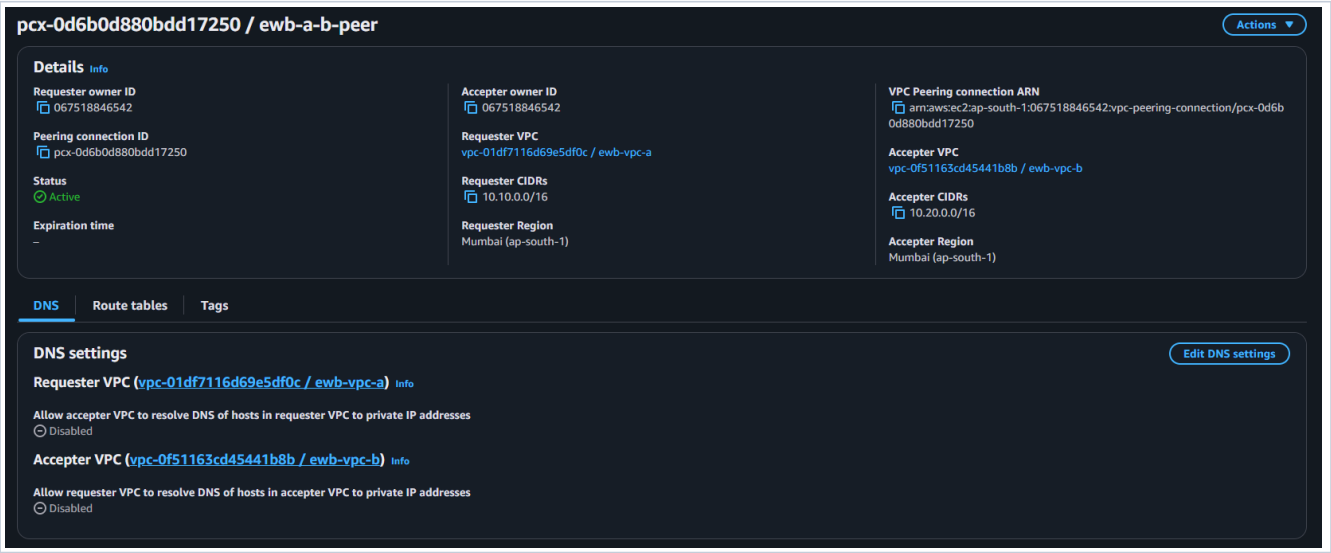


ewb-rt-a-private - associated with ewb-a-private. Routes: 10.10.0.0/16 local and 10.20.0.0/16 to the peering connection (App-A to VPC B). No internet default route. Before the peering route was added this table contained only the local route.



ewb-rt-b-private - associated with ewb-b-private. Routes: 10.20.0.0/16 local and 10.10.0.0/16 to the peering connection (return path to VPC A).

3. VPC peering connection



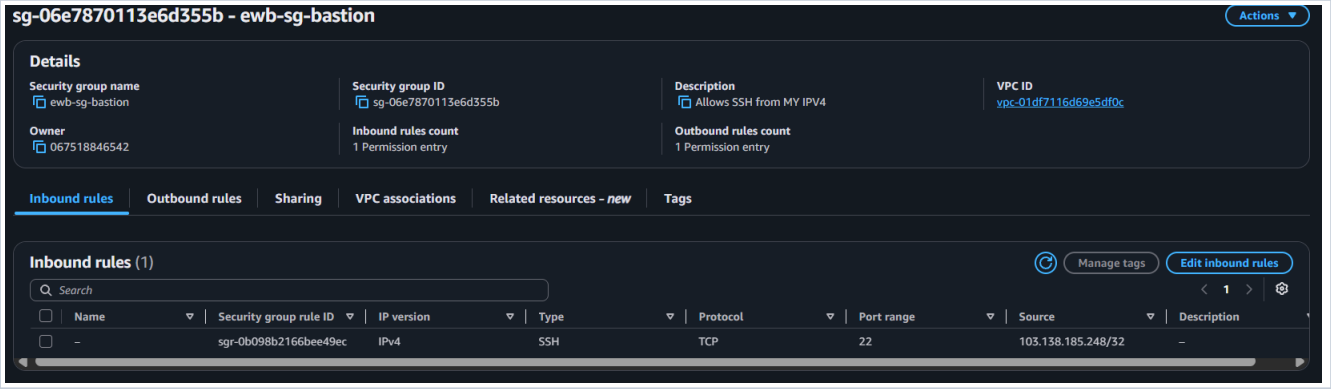
ewb-a-b-peer (pcx-0d6b0d880bdd17250) - Status: Active. Requester: ewb-vpc-a (10.10.0.0/16). Accepter: ewb-vpc-b (10.20.0.0/16). Same account, same Region (Mumbai).

Peering only creates the link. Traffic can use it only after route tables on both sides point to it and the security groups allow it.

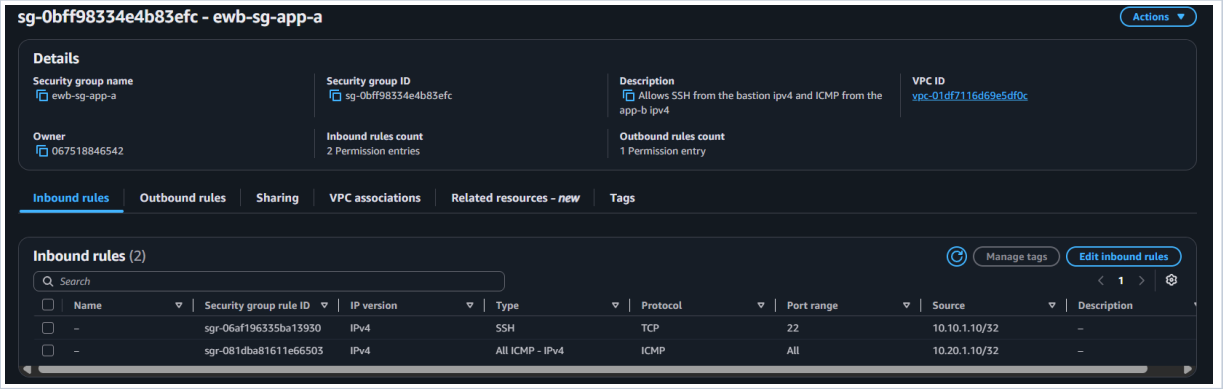
4. Security group inbound rules

Security group	Type / port	Source	Purpose
ewb-sg-bastion (VPC A)	SSH / TCP 22	103.138.185.248/32 (my public IPv4)	Admin access to bastion only from my IP
ewb-sg-app-a (VPC A)	SSH / TCP 22	10.10.1.10/32 (bastion)	SSH only from the bastion private IP
ewb-sg-app-a (VPC A)	ICMP IPv4	10.20.1.10/32 (App-B)	Allow ping from App-B
ewb-sg-app-b (VPC B)	SSH / TCP 22	10.10.1.10/32 (bastion)	SSH only from the bastion private IP
ewb-sg-app-b (VPC B)	ICMP IPv4	10.10.2.10/32 (App-A)	Allow ping from App-A

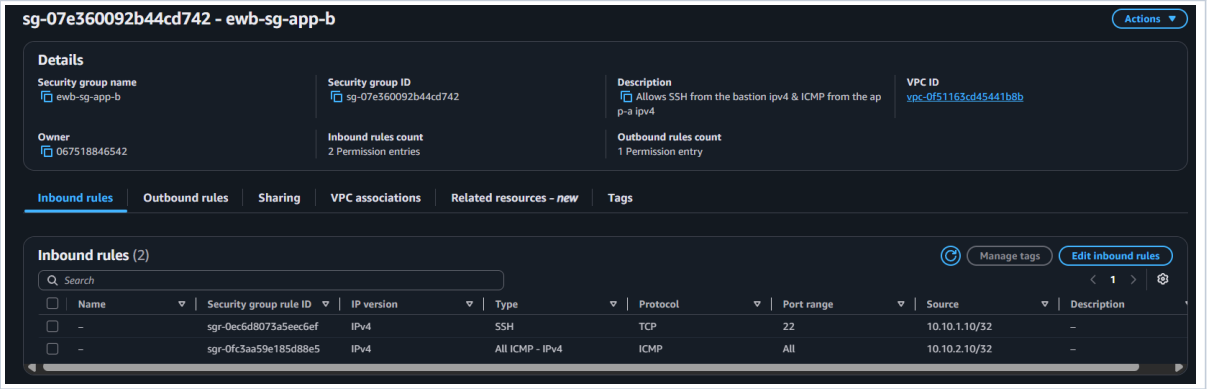
No rule uses 0.0.0.0/0. Outbound rules are left at the default allow-all; security groups are stateful, so replies are allowed automatically. Network ACLs are the default allow-all. The two ICMP rules are limited to a single /32 source host each; the console lists them as "All ICMP - IPv4".



ewb-sg-bastion (sg-06e7870113e6d355b) - one inbound rule: SSH TCP 22 from 103.138.185.248/32.

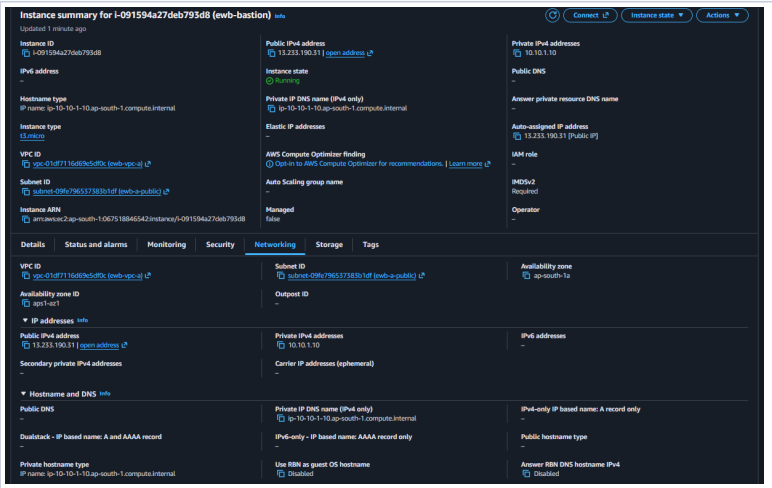


ewb-sg-app-a (sg-0bff98334e4b83efc) - inbound: SSH 22 from 10.10.1.10/32 (bastion) and ICMP from 10.20.1.10/32 (App-B).

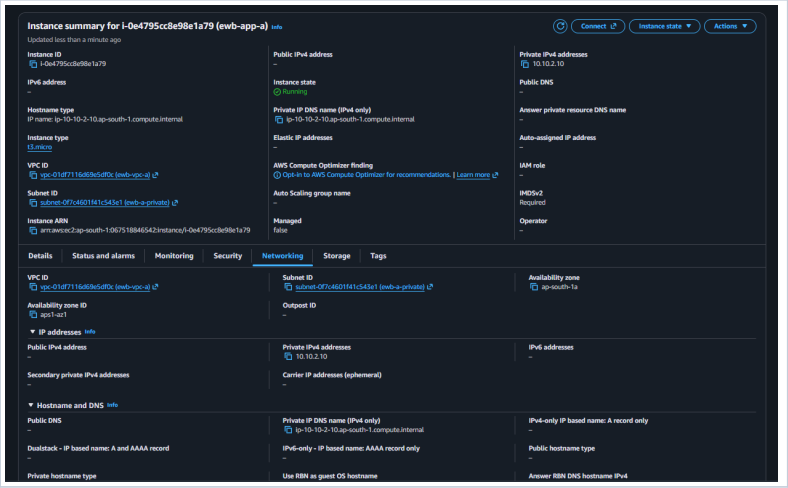


ewb-sg-app-b (sg-07e360092b44cd742) - inbound: SSH 22 from 10.10.1.10/32 (bastion) and ICMP from 10.10.2.10/32 (App-A).

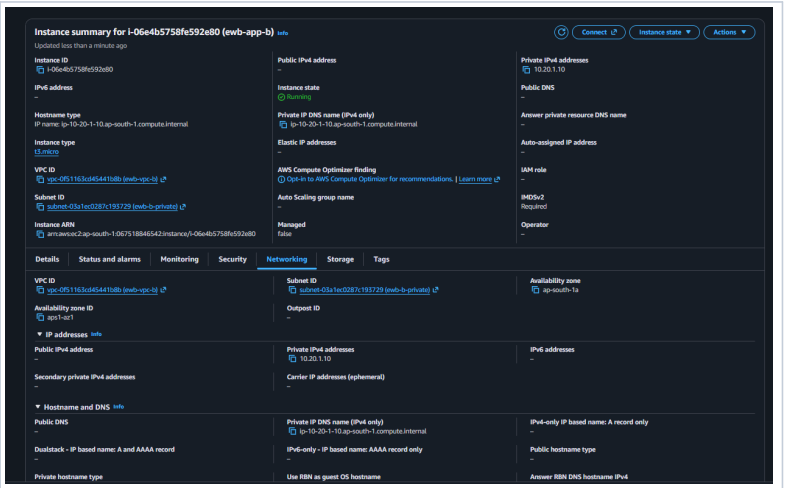
5. EC2 instances and isolation (Test 05)



ewb-bastion (i-091594a27deb793d8) - subnet ewb-a-public, private IP 10.10.1.10, public IPv4 13.233.190.31 (the only host with a public address).



Test 05 / Isolation - ewb-app-a (i-0e4795cc8e98e1a79): subnet ewb-a-private, private IP 10.10.2.10, Public IPv4 address empty. Result: no public IPv4.

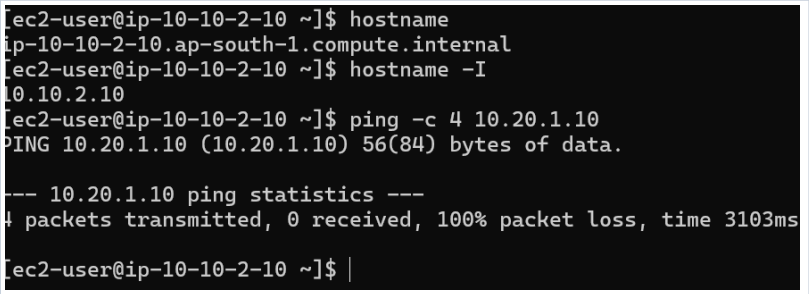


Test 05 / Isolation - ewb-app-b (i-06e4b5758fe592e80): subnet ewb-b-private, private IP 10.20.1.10, Public IPv4 address empty. Result: no public IPv4.

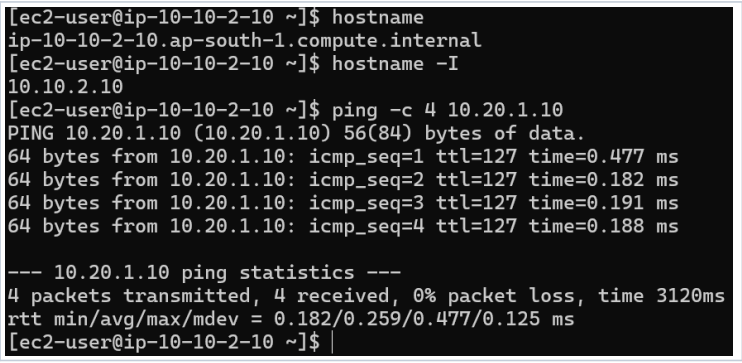
6. Connectivity tests

All tests use private destination IPs. Each terminal screenshot shows the host (from the shell prompt and `hostname` / `hostname -I`), the command and its result.

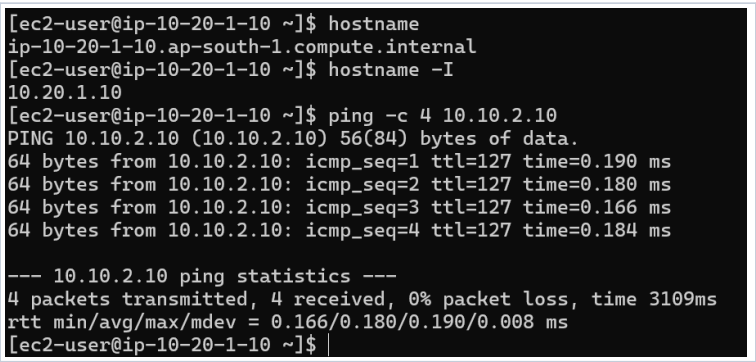
Test	Where / command	Expected	Result
01 Baseline	App-A: ping -c 4 10.20.1.10 (no peering path)	No replies	4 transmitted, 0 received, 100% packet loss
02 Forward path	App-A: ping -c 4 10.20.1.10 (after routes)	4 replies, 0% loss	4 received, 0% packet loss
03 Reverse path	App-B: ping -c 4 10.10.2.10 (after routes)	4 replies, 0% loss	4 received, 0% packet loss
04 Private SSH	Laptop: ssh ewb-app-b	Shell on App-B via bastion	Shell opened; login came from 10.10.1.10 (bastion)
05 Isolation	EC2 details of App-A and App-B	No public IPv4	Public IPv4 empty on both hosts



Test 01 / Baseline - host ewb-app-a (10.10.2.10): ping -c 4 10.20.1.10 gets no replies (100% packet loss). Result: baseline failure, as expected.



Test 02 / Forward path - host ewb-app-a (10.10.2.10): ping -c 4 10.20.1.10 returns 4 replies, 0% packet loss. Result: success.



Test 03 / Reverse path - host ewb-app-b (10.20.1.10): ping -c 4 10.10.2.10 returns 4 replies, 0% packet loss. Result: success.

```
Devops-engineer:~# ssh ewb-app-b
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Sat Oct 24 13:20:41 2026 from 10.10.1.10
[ec2-user@ip-10-20-1-10 ~]$
```

Test 04 / Private SSH - from my laptop (WSL) ssh ewb-app-b connects through the bastion; the prompt shows ip-10-20-1-10 and the last login was from 10.10.1.10 (the bastion private IP). Result: success.

7. Controlled failure and recovery

To prove that the security group (and not only the route) controls the traffic, I temporarily removed only the ICMP inbound rule (source 10.10.2.10/32) from ewb-sg-app-b and kept the SSH rule. The forward ping from App-A stopped. I then added the same ICMP rule back and the ping worked again.



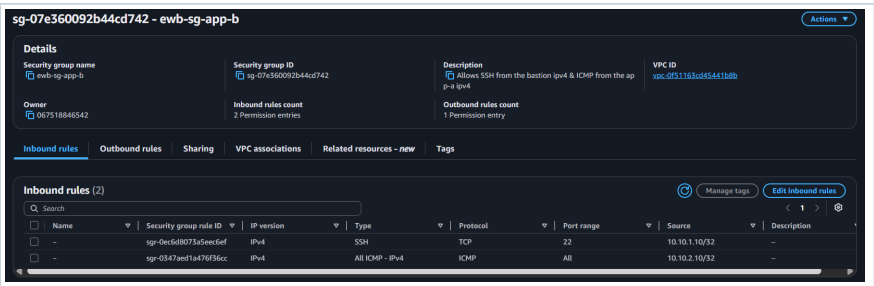
Step 1 - ewb-sg-app-b after removing the ICMP rule: only the SSH rule (10.10.1.10/32) remains (1 inbound rule).

```
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3110ms

[ec2-user@ip-10-10-2-10 ~]$
```

Step 2 - host ewb-app-a: ping -c 4 10.20.1.10 now fails, 4 transmitted, 0 received, 100% packet loss (failure).



Step 3 - ewb-sg-app-b after restoring the rule: SSH 22 from 10.10.1.10/32 and ICMP from 10.10.2.10/32 (2 inbound rules).

```
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.196 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=127 time=0.189 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=127 time=0.165 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=127 time=0.174 ms

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3127ms
rtt min/avg/max/mdev = 0.165/0.181/0.196/0.012 ms
[ec2-user@ip-10-10-2-10 ~]$
```

Step 4 - host ewb-app-a: ping -c 4 10.20.1.10 returns 4 replies, 0% packet loss (recovery).

8. Forward path and return path

Forward path (App-A to App-B). App-A (10.10.2.10) sends a packet to 10.20.1.10. The route table ewb-rt-a-private matches 10.20.0.0/16 and sends it to the peering connection pcx-0d6b0d880bdd17250 instead of the local route. In VPC B the packet reaches App-B, whose security group ewb-sg-app-b allows ICMP from 10.10.2.10/32.

Return path (App-B to App-A). The reply to 10.10.2.10 leaves App-B. The route table ewb-rt-b-private matches 10.10.0.0/16 and sends it back through the same peering connection. Because security groups are stateful, the reply is allowed automatically. Without this return route the request would arrive but the reply would have no path back, so one-way routes are not enough.

Administration path. Laptop to bastion over the internet gateway (SSH 22 allowed only from my public IP). With ProxyJump, the bastion then opens a separate SSH connection to App-A or App-B. For App-B this connection crosses the peering link, which is why ewb-rt-a-public has a 10.20.0.0/16 route to the peering connection, and why App-B accepts SSH only from the bastion's private IP (10.10.1.10/32) - App-B sees the bastion, not my laptop, as the source.

9. Questions to explain

1. Why is the return route needed? A route table only decides where traffic goes next. App-A's request reaches App-B through the peering route, but App-B's reply is addressed to 10.10.0.0/16, so ewb-rt-b-private must also have a route for that CIDR through the peering connection. Security groups are stateful and allow the reply, but they do not create a path for it. With a one-way route the reply has nowhere to go and the ping or SSH fails.

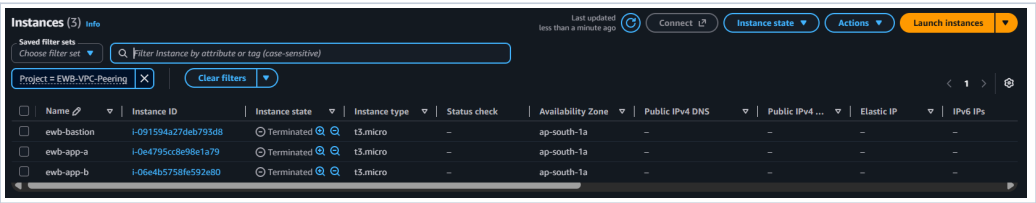
2. Why is the App-B SSH source the bastion's private IP? With ProxyJump my laptop only connects to the bastion. The bastion then opens a new SSH connection to App-B, so App-B sees the bastion's private address (10.10.1.10) as the source, not my laptop's public IP. The rule 10.10.1.10/32 therefore allows only the bastion to reach App-B. The login line in the Test 04 screenshot (last login from 10.10.1.10) confirms this.

3. Why does Active peering not guarantee a successful ping? The peering connection is only the link. Traffic also needs routes in both VPCs, a security group rule on the destination (and network ACLs that allow it). My controlled failure showed this: the peering and routes stayed Active, but removing the ICMP rule from ewb-sg-app-b made the ping fail until I restored it.

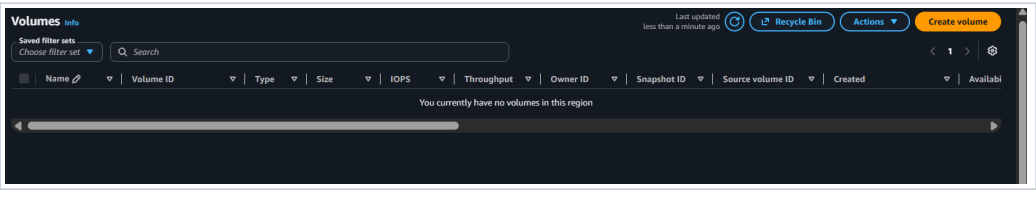
4. Can VPC B use VPC A's internet gateway over peering? No. Peering is one-to-one and non-transitive, and it does not forward traffic from one VPC out through the other VPC's internet gateway. VPC B has no internet gateway or public IPs, so it stays private; only the bastion in VPC A has a public address.

10. Cleanup evidence

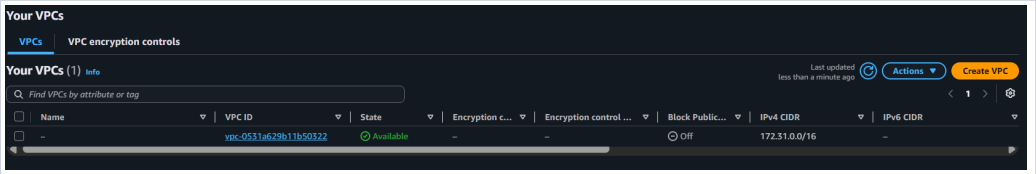
After saving all evidence, I removed every resource created for this project: the three EC2 instances and their volumes, the peering routes and the peering connection, the security groups, subnets, custom route tables, the internet gateway and both VPCs.



Cleanup 1 - EC2 instances filtered by tag Project = EWB-VPC-Peering: ewb-bastion, ewb-app-a and ewb-app-b are all in the Terminated state.



Cleanup 2 - EBS volumes in ap-south-1: "You currently have no volumes in this region" (no leftover lab volumes).



Cleanup 3 - Your VPCs in ap-south-1: only the account's default VPC (172.31.0.0/16) remains; ewb-vpc-a and ewb-vpc-b are deleted.

11. What I learned

I learned that a VPC peering connection is only a link: to actually use it I need non-overlapping CIDRs, routes on both sides (forward and return), and security group rules on the destination. I tested this myself by showing a failed ping before the routes existed, successful pings in both directions afterwards, and a controlled failure when I removed the ICMP rule. I also learned how a bastion host with SSH ProxyJump lets me reach private instances without copying the key onto any server, and why the private servers see the bastion's private IP as the SSH source. Finally, I practised keeping the servers private (no public IPs, no 0.0.0.0/0 rules) and cleaning up afterwards.